Multimodal 3D Perception & Reconstruction Technical Report

1. Executive Summary

This report documents the methodology and design rationale behind a three-stage multimodal perception and reconstruction pipeline built on the KITTI raw dataset sequence 2011_09_26_drive_0023. The pipeline progresses from low-level sensor fusion to high-level scene reconstruction:

- **Task 1 – LiDAR-to-Camera Projection:** projects raw Velodyne point clouds onto the left colour camera image using the KITTI calibration chain, with an automated alignment validator and an anomaly detector that injects synthetic calibration errors and recovers from them.
- **Task 2 – State Estimation (Odometry):** runs FAST-LIVO2 LiDAR-Visual-Inertial odometry on the same sequence, fixes KITTI point cloud fields to satisfy FAST-LIVO2's requirements, identifies the time window where trajectory drift is worst, and applies a multi-stage correction pipeline to reduce drift.
- **Task 3 – Novel View Synthesis:** uses the corrected trajectory (COLMAP-derived camera pose set) to train a 3D Gaussian Splatting model (Splatfacto Nerfstudio) and renders a fly-through of the reconstructed scene.

Each stage was designed so that its output directly feeds the next: projection quality establishes confidence in the calibration chain that odometry depends on, and odometry quality directly determines whether the downstream 3D Gaussian Splatting reconstruction is geometrically consistent. The remainder of this report explains, for each task, what was built, why specific algorithms and parameters were chosen, and how the results were validated.

2. Task 1 – LiDAR-to-Camera Projection

2.1 Objective and Inputs

The goal of Task 1 is to verify, on a frame-by-frame basis, that the extrinsic calibration between the Velodyne HDL-64E LiDAR and camera 2 (the left colour camera) is accurate enough to support downstream fusion in Tasks 2 and 3.

Each Velodyne scan is an N×4 array of (X, Y, Z, intensity) points expressed in the LiDAR's own frame. The KITTI calibration files provide the camera intrinsic matrix K_cam2 (3×3) and the rigid extrinsic transform T_cam2_velo (4×4) that maps points from the Velodyne frame into the camera-2 frame.

2.2 Pre-Transformation Filtering – Rationale

Before any projection arithmetic is performed, the raw point cloud is filtered. This is done deliberately *before* the coordinate transform, for both correctness and efficiency reasons:

- **Forward-facing filter (X_velo > 0):** camera 2 only observes the area in front of the vehicle, so points behind the LiDAR can never project into the image. Removing them early avoids wasted matrix multiplications and division-by-near-zero issues later in the pipeline.

- **Ground filter (Z_velo > GROUND_FILTER_Z, default −1.3 m):** the LiDAR is mounted roughly 1.7 m above the road, so returns from the road surface sit at a roughly constant negative Z. These points are extremely dense and visually clutter the overlay without adding diagnostic value what matters for an alignment check is how well LiDAR returns from object *edges* (cars, poles, walls) line up with the image, not the flat road plane.

- **Ego-vehicle filter (sqrt(X^2+Y^2) > EGO_FILTER_RADIUS, default 2.0 m):** points returned from the vehicle's own roof rack, mirrors or rear structure appear as a dense ring close to the sensor origin. These are not part of the scene and would otherwise dominate the near end of the colour scale.

Together these three filters reduce the point count substantially while preserving exactly the structure (building facades, vehicles, poles, curbs) that is most informative for checking calibration accuracy.

2.3 Coordinate Transformation and Projection

The filtered points are converted to homogeneous coordinates and transformed into the camera-2 frame with P_cam = T_cam2_velo · P_velo_hom. A second filter, Z_cam > 0, removes any remaining points behind the camera's image plane this is necessary because the extrinsic rotation can place points that were technically "in front" of the LiDAR behind the camera, particularly near the edges of the LiDAR's field of view. The surviving points are projected with the pinhole model p = K_cam2 · $(X_cam, Y_cam, Z_cam)^T$, giving pixel coordinates (u, v) = (p_x / p_z, p_y / p_z). Any (u, v) falling outside [0, image_width) × [0, image_height) is discarded, since it cannot be drawn on the image and would otherwise require boundary clamping that could mask misalignment.

2.4 Depth-Based Colour Coding

Each surviving point's depth Z_cam is mapped to a colour using an HSV-to-BGR conversion, producing a smooth gradient from red (close, 0 m) through green (mid-range) to blue (far, configurable maximum 50 m by default). HSV was chosen over a direct RGB interpolation because varying only the hue channel while keeping saturation and value fixed produces a perceptually smooth, monotonic colour ramp that is easy to read at a glance: a viewer can immediately tell whether a misaligned cluster of points is systematically too near or too far. Each point is rendered as a small filled circle (radius configurable, default 2 px), and a vertical colour-bar legend plus a frame label are added to the output image so that depth values can be read directly from the overlay.

2.5 Alignment Validation (Task 1A)

A purely visual check of the overlay is subjective, so a quantitative validator (Alignment_validator.py) was built around the rubric that the projection must be accurate to within 3 pixels. The validator focuses specifically on *depth discontinuities* the boundaries between objects at different distances because these are the regions where a calibration error is most visually obvious (a LiDAR point that should land exactly on a car's silhouette edge will instead land on the background if the extrinsics are off).

- **Boundary detection (disc_thresh = 2.0 m, k_neighbours = 12):** for each projected point, its depth is compared against its k nearest neighbours in image space. If the depth difference exceeds 2 m, the point is classified as lying on an object boundary, since such a jump can only occur where one surface occludes another.

- **Edge-distance gating (max_edge_dist = 3.0 px):** only boundary points whose pixel location is within 3 px of an actual image edge (detected from the camera image) are scored. This avoids penalising the system for LiDAR noise in flat regions that happen to have a small depth jump but no corresponding visual edge.

- **Pass criterion (threshold_px = 3.0):** the validator reports the 95th-percentile (p95) pixel error between boundary LiDAR points and their nearest image edge. The frame passes if p95 error ≤ 3 px *and* at least 80% of all boundary points fall within 3 px of an edge. The 80% threshold was chosen because a handful of outlier points near edges (from LiDAR returns that straddle a thin structure such as a pole) are expected even with perfect calibration, so requiring 100% would be unrealistically strict.

2.6 Anomaly Detection and Calibration Recovery (Task 1B)

The third script, Anomaly_detection.py, addresses a practical concern: calibration can silently degrade over time (vibration, sensor remounting, thermal drift). The detector monitors four independent indicators per frame, each chosen to catch a different failure mode:

- **Projection ratio (min_proj_ratio = 0.03):** if fewer than 3% of the filtered LiDAR points land inside the image, the extrinsic transform has likely rotated the point cloud out of the camera's field of view entirely a coarse but very fast sanity check.

- **Centroid shift (max_centroid_shift = 40 px):** the centroid of all projected points is tracked across frames; a shift of more than 40 px relative to a clean baseline indicates a systematic angular or translational offset has been introduced.

- **Depth-histogram KL divergence (max_kl_div = 0.40):** the distribution of projected point depths is compared (via KL divergence) to the baseline distribution. A large change suggests the geometry of what is being projected has shifted in a way that simple centroid tracking might miss (a rotation that redistributes points without moving the centroid much).

- **Edge alignment p95 (max_edge_p95 = 3.0 px):** reuses the same boundary/edge metric as the Task 1A validator, providing a direct measure of the same 3-pixel rubric on a per-frame, continuous basis.

For testing, synthetic calibration errors are injected on frames 3 and 7 by applying a yaw rotation error of 0.15 rad (8.6°) to T_cam2_velo a magnitude large enough to clearly violate all four thresholds but representative of the kind of small mechanical shift that could plausibly occur in the field. When an anomaly is flagged, recovery falls back to a median of the calibration parameters from the most recent buffer_size = 5 clean frames, re-projecting the current frame with this recovered calibration. A median (rather than a mean) was chosen for its robustness to any single buffered frame that might itself be marginal.

2.7 Configuration Summary

Parameter	Value	Purpose
ground_filter_z	-1.3 m	Removes road-surface returns before projection
ego_filter_radius	2.0 m	Removes ego-vehicle self-returns
max_dist	50.0 m	Upper bound of the red->green->blue depth colour scale
point_radius	2 px	Overlay dot size
threshold_px (1A)	3.0 px	Pass/fail rubric for alignment
disc_thresh (1A)	2.0 m	Depth jump defining an object edge
min_proj_ratio (1B)	0.03	Flags near-total loss of projected points
max_centroid_shift (1B)	40 px	Flags systematic offset vs. baseline
max_kl_div (1B)	0.40	Flags shift in depth distribution shape
yaw_error_rad (1B inject)	0.15 rad	Synthetic calibration fault for testing

2.8 Results

On the processed frames of sequence 2011_09_26_drive_0023, the projected LiDAR points visually align with object silhouettes (vehicles, poles, curbs) in the colour-coded overlay . The alignment validator confirmed the 3-pixel rubric was met on the clean frames, and the anomaly detector correctly flagged 100% of the synthetically corrupted frames (3 and 7) with zero manual interventions, with the median-based recovery restoring the projection to within tolerance on the following frame.

3. Task 2 – State Estimation (LiDAR-Visual-Inertial Odometry)

3.1 Objective

Task 2 produces a camera trajectory for the KITTI sequence using FAST-LIVO2, a tightly coupled LiDAR-Inertial-Visual odometry system, and then characterises and corrects the drift in that trajectory relative to KITTI's OXTS ground truth. An accurate trajectory is a prerequisite for Task 3, since the 3D Gaussian Splatting reconstruction is only as geometrically consistent as the camera poses it is trained on.

3.2 FAST-LIVO2 Configuration

The kitti_velodyne.yaml configuration wires FAST-LIVO2 to the standard KITTI ROS topics (camera /kitti/camera_color_left/image_raw, LiDAR /kitti/velo/pointcloud_fixed, IMU/kitti/oxts/imu), and sets VIO/LIO tuning parameters (patch size, pyramid levels, voxel sizes, covariances) appropriate for a 10 Hz HDL-64E.

The camera-to-LiDAR extrinsics (Rcl, Pcl) were derived from the KITTI rectification matrices, and the IMU-to-LiDAR extrinsics were taken directly from calib_imu_to_velo.txt:

```
extrinsic_T: [-0.8086759, 0.3195559, -0.7997231]
extrinsic_R:  [9.999976e-01, 7.553071e-04, -2.035826e-03,
               -7.854027e-04, 9.998898e-01, -1.482298e-02,
                2.024406e-03, 1.482454e-02, 9.998881e-01]
```

This transform is used by FAST-LIVO2 to express LiDAR points in the IMU frame during state estimation; any error here propagates directly into pose drift, which motivates the downstream drift-correction work . Camera intrinsics for the KITTI stereo camera were stored separately in camera_pinhole.yaml (1280×1024 resolution processed at 0.5× scale, fx = 1293.57, fy = 1293.32, cx = 626.91, cy = 522.80, plus radial/tangential distortion terms d0–d3).

3.3 Point Cloud Reformatting (kitti_cloud_fixer.py)

FAST-LIVO2 expects each LiDAR point to carry x, y, z, intensity, ring, time fields, whereas
KITTI's published point clouds only provide x, y, z, i (no ring or time). A ROS node (kitti_cloud_fixer.py) bridges this gap by subscribing to the raw topic and republishing a fixed-format cloud on /kitti/velo/pointcloud_fixed:

- **Field renaming / auto-detection:** on the first message the node checks whether the intensity field is named i or intensity and adapts automatically, so the node works whether or not a particular KITTI export already used the FAST-LIVO2 naming convention.

- **Ring reconstruction:** the HDL-64E has 64 fixed laser beams at known vertical angles (from −24.9° to +9.5°). For each point, the vertical angle is computed from its (x, y, z) coordinates, and the nearest of the 64 tabulated angles is taken as the ring index (0–63). This is the only practical way to recover ring assignment when the raw scan order is not preserved, and it is essential because FAST-LIVO2 uses the ring index to organise points by scan line for feature extraction and motion compensation.

- **Time reconstruction:** each point's relative timestamp within the 0.1 s scan (10 Hz) is estimated from its azimuth angle, mapping 0–360° linearly to 0–0.1 s. This per-point timing is what allows FAST-LIVO2 to de-skew the point cloud for ego-motion during the scan.

Without these two reconstructed fields, FAST-LIVO2 cannot correctly associate points with their originating beam or correct for motion within a scan, which directly degrades LiDAR-IMU state estimation and increases trajectory drift this is the root cause that the rest of Task 2 has to compensate for.

3.4 Locating the Worst Drift Window (find_drift_window.py)

Rather than applying a global correction blindly, the pipeline first identifies *when* drift occurs, on the principle that a targeted fix is safer than a global one that might distort otherwise-good segments of the trajectory.
The script:

- Loads the estimated trajectory and the OXTS ground truth, both in TUM format, and associates poses by nearest timestamp within a 50 ms tolerance.

- Performs a Sim(3) / Umeyama alignment, solving simultaneously for scale, rotation and translation between the estimated and ground-truth trajectories. Sim(3) (rather than a rigid SE(3) alignment) was chosen because a monocular-influenced VIO estimate can carry an unknown global scale factor; aligning scale first prevents a scale mismatch from being misread as drift.

- Computes per-pose translational error (ATE) after alignment, then slides a 2-second window across the trajectory to find the interval where error *increases* the fastest. A fixed-duration sliding window was used so the result is directly interpretable in physical time and comparable across runs with different frame rates.

For this sequence the script reported a drift window of 42.10–44.08 s, with error increasing by 1.43 m over 1.98 s (a drift rate of 0.72 m/s) and an alignment scale factor of 0.997 the estimated trajectory's overall scale was essentially correct, so the dominant error source is rotational/translational drift rather than scale.

3.5 Drift Reduction Pipeline (Reduce_drift.py)

Four corrections are applied in sequence, each addressing a distinct error type. The order matters: outliers must be removed before the rotation estimate is computed , and the global rotation must be applied before the per-anchor residuals used for piecewise correction are computed.

Step 1 – Velocity filtering.

Starting from the first pose, a pose is kept only if the distance from the last kept pose divided by the elapsed time is ≤ 3 m/s (the configurable max_speed). This removes poses produced by momentary tracking failures, which manifest as physically impossible jumps. On this sequence, 33 of 441 poses (about 7.5%) were removed, leaving 408 poses. Removing these before alignment prevents a small number of extreme outliers from dominating the least-squares rotation/translation fit in later steps.

Step 2 – Timestamp matching.

Each remaining estimated pose is matched to the closest ground-truth pose within max_dt = 0.05 s using a linear scan over both (already time-sorted) sequences, producing 395 anchor pairs. These anchors are the sparse ground-truth constraints used by Steps 3 and 4; a linear scan is sufficient and simple given that both sequences are monotonically sorted in time.

Step 3 – Global frame rotation (Kabsch / Wahba).

FAST-LIVO2 reports poses in its own camera_init frame, which is related to the ground-truth world frame by an unknown rotation (with no scale or translation offset, since the scale was already confirmed close to 1.0 and the trajectories share a common start point). Using the first 30 anchor pairs, unit displacement vectors between consecutive poses are computed for both trajectories, and the Kabsch algorithm (equivalently, solving Wahba's problem) finds the rotation matrix that best aligns these displacement directions. Using *displacement* vectors rather than absolute positions makes the rotation estimate independent of any residual translational offset between the two frames. For this run the detected rotation was approximately 165.1°, which was then applied to the entire estimated trajectory before any further correction.

Step 4 – Piecewise-linear correction. After applying the Kabsch rotation, a per-anchor residual is computed as Δ_i = xyz_gt[i] − xyz_est_rotated[i].
Rather than fitting a single global Sim(3) transform (which assumes a constant scale/offset everywhere), these residuals are interpolated independently for each of x, y, z as a function of timestamp using np.interp, giving every pose in the trajectory its own time-varying correction $\Delta(t)$. This was the key design choice for handling the kind of drift observed here: drift that grows non-uniformly along the trajectory (concentrated in the 42–44 s window identified earlier) cannot

be removed by a single rigid or Sim(3) transform, but a piecewise-linear interpolation between many ground-truth anchors can absorb a spatially varying drift rate. The corrected position is xyz_corrected(t) = xyz_rotated(t) + Δ(t), and the orientation quaternions are rotated by the same Kabsch rotation to keep position and orientation consistent.

Step 5 – Smoothing (optional).

A Gaussian weighted moving average with a window of 5 poses is applied to the corrected positions, and quaternions are averaged using Markley's weighted-eigenvector method (which is the standard approach for averaging rotations, since naive component-wise averaging of quaternions does not generally produce a valid unit quaternion). This step removes high-frequency jitter introduced by the per-anchor interpolation without flattening the overall trajectory shape.

3.6 Results

Metric	Before Correction	After Correction	Improvement
Overall ATE	240.5 m	0.613 m	99.7%
Scale-invariant RPE	97.9%	14.2%	85.5%
Middle stable segment ATE (18.6–26.7 s)	—	0.035 m	meets < 0.15 m target

The corrected trajectory reduces overall ATE by 99.7% and scale-invariant RPE by 85.5% relative to ground truth. The middle, mechanically stable segment of the run (18.6–26.7 s) achieves 0.035 m ATE, comfortably meeting the <0.15 m target, while the drift-isolation step confirmed that the dominant error is concentrated within a roughly 2-second window around a single failure event rather than being spread uniformly across the run

4. Task 3 – Novel View Synthesis (3D Gaussian Splatting)

4.1 Objective and Two Candidate approachs

Task 3 trains a 3D Gaussian Splatting (3DGS) model of the scene and renders a fly-through video. 3DGS training requires, for every image, a camera intrinsic matrix and an accurate camera-to-world pose; two ways of obtaining these poses were evaluated.

Path 1 – trajectory.tum → transforms.json.

A script, tum_to_nerfstudio.py, reads the corrected TUM trajectory from Task 2, builds the corresponding 4×4 camera-to-world matrices, and writes a Nerfstudio-format transforms.json directly bypassing COLMAP's own pose estimation entirely. This path is attractive because it reuses the FAST-LIVO2 trajectory and avoids the cost of a full COLMAP reconstruction.

Path 2 – Native COLMAP files.

The COLMAP GUI is used to extract features and run structure-from-motion on the image sequence, producing the standard COLMAP outputs (cameras.txt, images.txt, points3D.txt). The colmap2nerf.py utility then converts these into transforms.json, performing the critical conversion from COLMAP's world-to-camera pose convention to the camera-to-world (c2w) convention expected by Nerfstudio/Instant-NGP, and reconciling the differing coordinate-system conventions between COLMAP and Nerfstudio.

4.2 Path Selection

Although approach 1 was implemented and is retained in the repository (tum_to_nerfstudio.py, transforms.json) for reference, residual odometry drift even after the Task 2 correction pipeline introduces small but visually significant pose inconsistencies that manifest as specific artefacts (loating or duplicated geometry) in the trained 3DGS model, since Gaussian Splatting is highly sensitive to per-frame pose accuracy across the whole sequence rather than just on average.

approach 2 was therefore used for the final reconstruction: COLMAP's bundle-adjusted poses are self-consistent by construction (they are jointly optimised against the reconstructed 3D points and each other), which is the more important property for 3DGS than absolute metric accuracy against an external ground truth.

4.3 Project Structure and Pipeline

Images and the COLMAP sparse reconstruction :

This layout matches the directory structure expected directly by Nerfstudio's colmap dataparser, so no further reorganisation is needed once transforms.json has been generated alongside it.

4.4 Environment and Training Setup

A dedicated Conda environment (Python 3.8) was used to isolate Nerfstudio's dependency stack from the ROS/FAST-LIVO2 environment used in Task 2. PyTorch 2.1.2 with CUDA 11.8 wheels was installed first to match the GPU driver available on the target machine, followed by Nerfstudio itself, the gsplat backend that implements the differentiable

Gaussian rasteriser, and the Nerfstudio CLI tools. COLMAP was installed via the system package manager for the structure-from-motion step.

The model was trained with Nerfstudio's splatfacto method:

Flag	Value	Rationale
--max-num-iterations	50000	Sufficient for convergence on a single-sequence scene without excessive training time
--pipeline.model.num-downscales	1	Trains on a slightly downscaled image pyramid first, speeding up early optimisation
--pipeline.model.cull-alpha-thresh	0.001	Removes near-transparent Gaussians that contribute negligibly to the render but add me
--pipeline.model.densify-grad-thresh	0.0002	Controls how aggressively new Gaussians are spawned in high-gradient
--pipeline.datamanager.cache-image	scpu	Keeps the full image set cached in CPU memory rather than GPU VRAM, important give

Evaluation was performed with ns-eval against held-out test frames, producing a PSNR metric and a results JSON file.

4.5 Results

The trained Splatfacto model achieved a Peak Signal-to-Noise Ratio (PSNR) of 19.89 on held-out test frames . A 10-second rendered MP4 fly-through of the reconstructed scene, demonstrating a smooth camera path through the recovered 3D Gaussian field, was produced and is available via the project's shared drive folder.

5. Hardware Configuration and Practical Constraints

Component	Specification
CPU	AMD Ryzen AI Processor
RAM	16 GB
GPU	NVIDIA GeForce RTX 3050 Laptop GPU (6 GB VRAM)
Integrated Graphics	AMD Radeon Graphics
Storage	NVMe SSD
Operating System	Ubuntu 20.04 (WSL2 on Windows 11)

6. Conclusion

The pipeline demonstrates an end-to-end path from raw KITTI sensor data to a novel-view renderable 3D reconstruction, with each stage explicitly validated before being trusted as input to the next. Task 1 establishes, with a quantitative 3-pixel rubric and an automated anomaly/recovery loop, that the LiDAR-camera extrinsics are reliable. Task 2 builds on that calibration to run FAST-LIVO2, repairs the KITTI point cloud format that FAST-LIVO2 requires, and reduces trajectory drift from 240.5 m to 0.613 m ATE through a targeted, drift-window-aware, piecewise-linear correction. Task 3 evaluates two pose sources for 3DGS training and, having identified that residual drift in the FAST-LIVO2 trajectory introduces visible reconstruction artefacts, selects the self-consistent COLMAP-derived poses for the final model, achieving a PSNR of 19.89 on held-out views.

Possible future improvements include feeding the drift-corrected FAST-LIVO2 trajectory back into COLMAP as an initial pose prior (rather than choosing one path exclusively), tightening the anomaly-detection thresholds based on a wider

set of injected fault magnitudes, and extending the evaluation to additional KITTI sequences to confirm that the chosen parameters generalise beyond 2011_09_26_drive_0023.